

ML Lab Week 14: CNN Image Classification Report

Name: Jeel Nada

SRN: PES2UG23CS357

Date: 19-11-2025

1. Introduction

In this lab, the objective was to design, build, and train a Convolutional Neural Network (CNN) using PyTorch. The goal of the model was to accurately classify images of hand gestures into one of three categories: 'rock', 'paper', or 'scissors', utilizing the Kaggle "Rock Paper Scissors" dataset. The process involved implementing data loading pipelines, defining a deep learning architecture, and evaluating the model on unseen test data.

2. Model Architecture

CNN Architecture

- **Convolutional Layers:** The model utilizes a stack of 3 convolutional layers.
 1. **Layer 1:** Input 3 channels (RGB) $\rightarrow$ 16 output channels (Kernel: 3x3, Padding: 1).
 2. **Layer 2:** 16 input channels $\rightarrow$ 32 output channels (Kernel: 3x3, Padding: 1).
 3. **Layer 3:** 32 input channels $\rightarrow$ 64 output channels (Kernel: 3x3, Padding: 1).
 - **Activation Functions:** The Rectified Linear Unit (ReLU) activation function is applied after every convolutional layer and the first fully connected layer to introduce non-linearity.
 - **Pooling:** Max Pooling layers (MaxPool2d) with a kernel size of 2 and stride of 2 are applied after every convolutional block. This effectively halves the spatial dimensions of the feature maps at each stage ($128 \times 128 \rightarrow 64 \times 64 \rightarrow 32 \times 32 \rightarrow 16 \times 16$).
-

Fully-Connected Classifier

- **Structure:** The classifier consists of a flattened input followed by two linear layers.
 1. **Flatten:** The 64 feature maps of size 16x16 are flattened into a vector of size 16,384 ($64 * 16 * 16$).
 2. **Hidden Layer:** A Linear layer maps 16,384 features to 256 neurons, followed by a ReLU activation and a Dropout layer (probability 0.3) to prevent overfitting.
 3. **Output Layer:** A final Linear layer maps the 256 features to 3 output classes (Rock, Paper, Scissors).
-

3. Training and Performance

Hyperparameters

- **Optimizer:** Adam (`torch.optim.Adam`)
- **Loss Function:** CrossEntropyLoss (`nn.CrossEntropyLoss`)
- **Learning Rate:** 0.001
- **Number of Epochs:** 10

Results

- **Final Test Accuracy:** 98.17%
-

4. Conclusion and Analysis

Discussion

The CNN model performed effectively on the dataset. By using three convolutional layers, the network was able to learn hierarchical features—starting from simple edges in the first layer to complex hand shapes in the deeper layers. The use of Max Pooling significantly reduced the computational load while maintaining spatial invariance. The inclusion of the Dropout layer in the classifier helped regularize the model, preventing it from memorizing the training data too closely.

Future Improvements

1. **Data Augmentation:** Currently, the model only resizes and normalizes images. Adding transformations such as `RandomHorizontalFlip` and `RandomRotation` would make the model more robust to different hand orientations.
2. **Learning Rate Scheduler:** Implementing a scheduler (e.g., `StepLR`) to decrease the learning rate as training progresses could help the model converge to a better global minimum and improve final accuracy.